ARTIFICIAL INTELLIGENCE

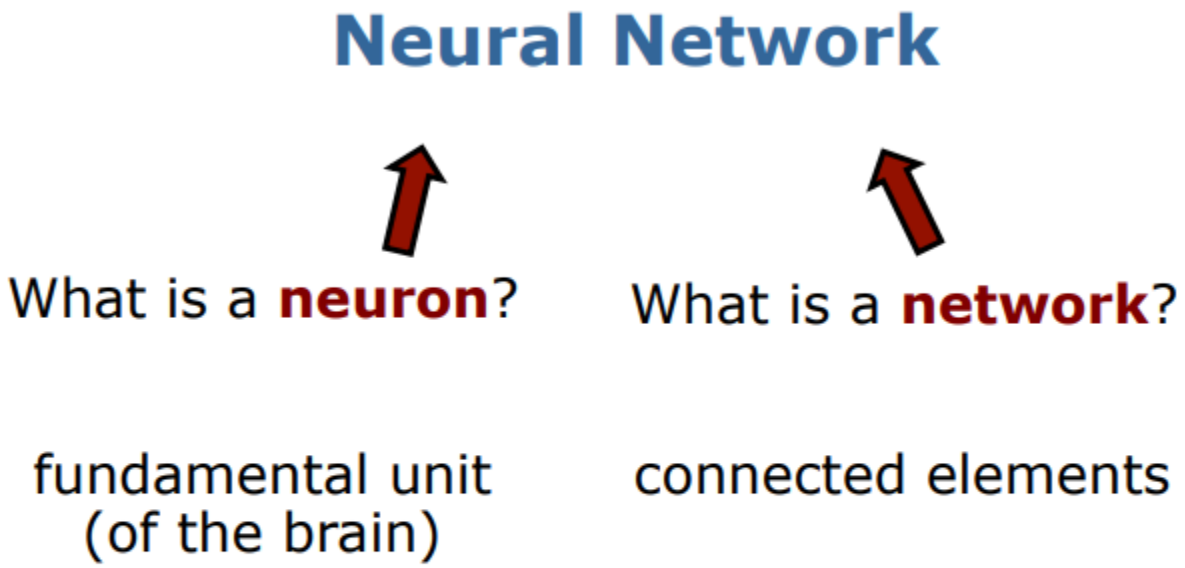
Lecture Set 19
Neural Networks

Dr. Maria Siddiqua
Assistant Professor
Department of AI & DS
FAST-NUCES Karachi
maria.siddiqua@nu.edu.pk

Course Contents

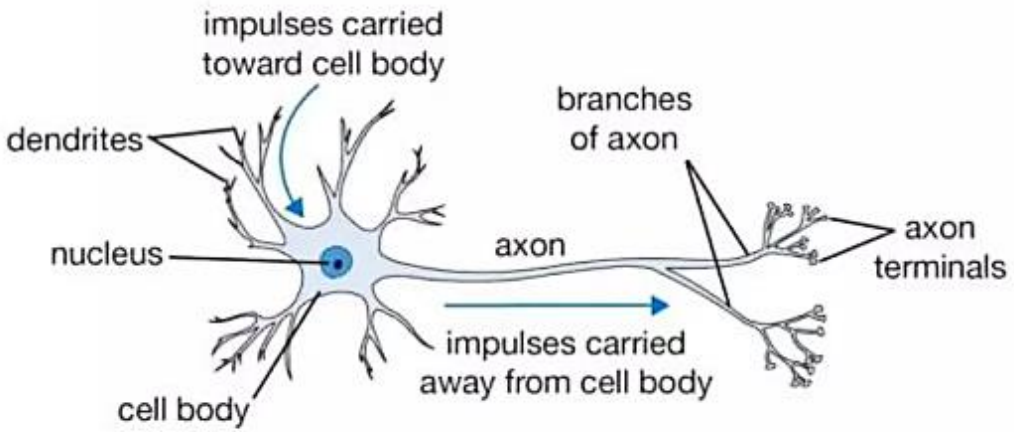
1. Introduction to AI
2. Agents and Environments
3. Problem Solving by Searching
4. Uninformed Search
5. Informed Search
6. Local Search
7. Constraint Satisfaction Problems
8. Adversarial Search
9. Knowledge Representation and Logical Agents
10. Uncertainty, Conditional Probability, Bayes Rule
11. Bayesian Networks, Naïve Bayes Classifier
12. Hidden Markov model, Markov Decision Process
13. **Machine Learning (Supervised, Unsupervised, Reinforcement Learning)**

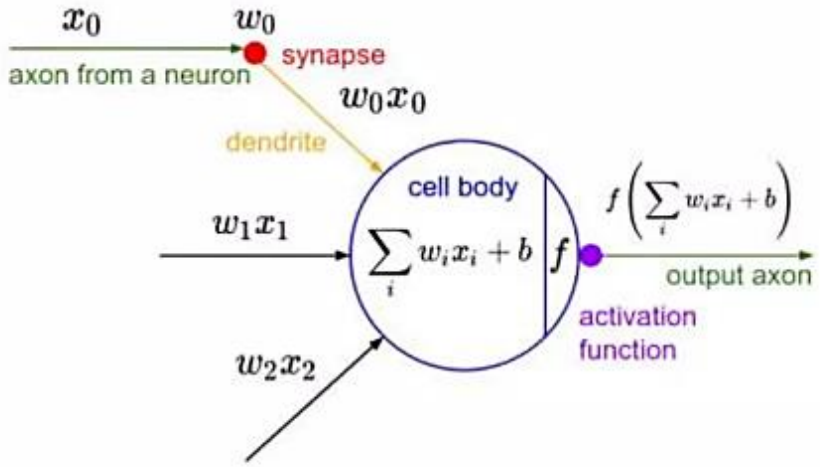
Introduction to Neural Networks



Biological neuron vs artificial neuron

Fundamental units of the brain that: - Receive sensory input from the external world or from other neurons - Transform and relay signals - Send signals to other neurons and also motor commands to the muscles	Fundamental units of artificial neural networks that: - Receive inputs - Transform information - Create an output





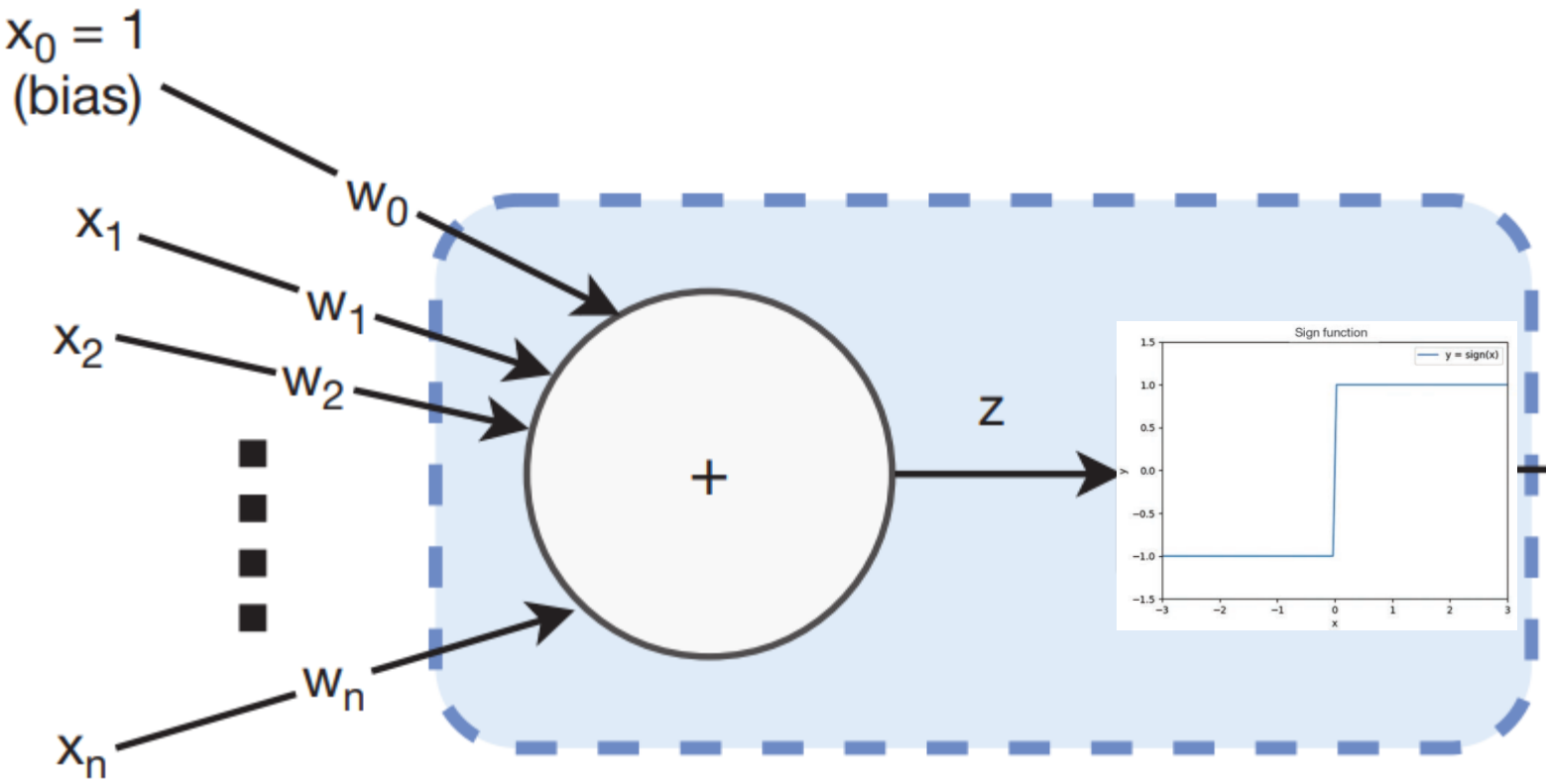
Complex	Single layer network
Unknown processing	Weighted sums and activation functions
Neuroplasticity	Connections remain same

Inspired not similar!

Perceptron

- A perceptron is a basic building block of artificial neural networks.
- It serves as a simple model of a biological neuron.
- It was introduced by Frank Rosenblatt in 1957.
- The perceptron is particularly useful for binary classification problems, where it can learn to make decisions based on input features.

Perceptron



$$y = f(z), \text{ where}$$

$$z = \sum_{i=0}^{n} w_i x_i$$

$$f(z) = \begin{cases} -1, & z < 0 \\ 1, & z \geq 0 \end{cases}$$

$$x_0 = 1 \text{ (bias term)}$$

Key Components

- **Inputs (x0, x1, ..., xn):** A perceptron takes multiple binary inputs (-1 or 1) or real-valued inputs. Each input is associated with a weight (w1, w2, ..., wn), which represents the strength of the connection between the input and the perceptron.
- **Weights (w0, w1, ..., wn):** Weights are parameters that the perceptron learns during the training process. They determine the influence of each input on the perceptron's output. Larger weights indicate a stronger influence.
- **Summation Function (∑):** The perceptron computes the weighted sum of its inputs using the following formula:

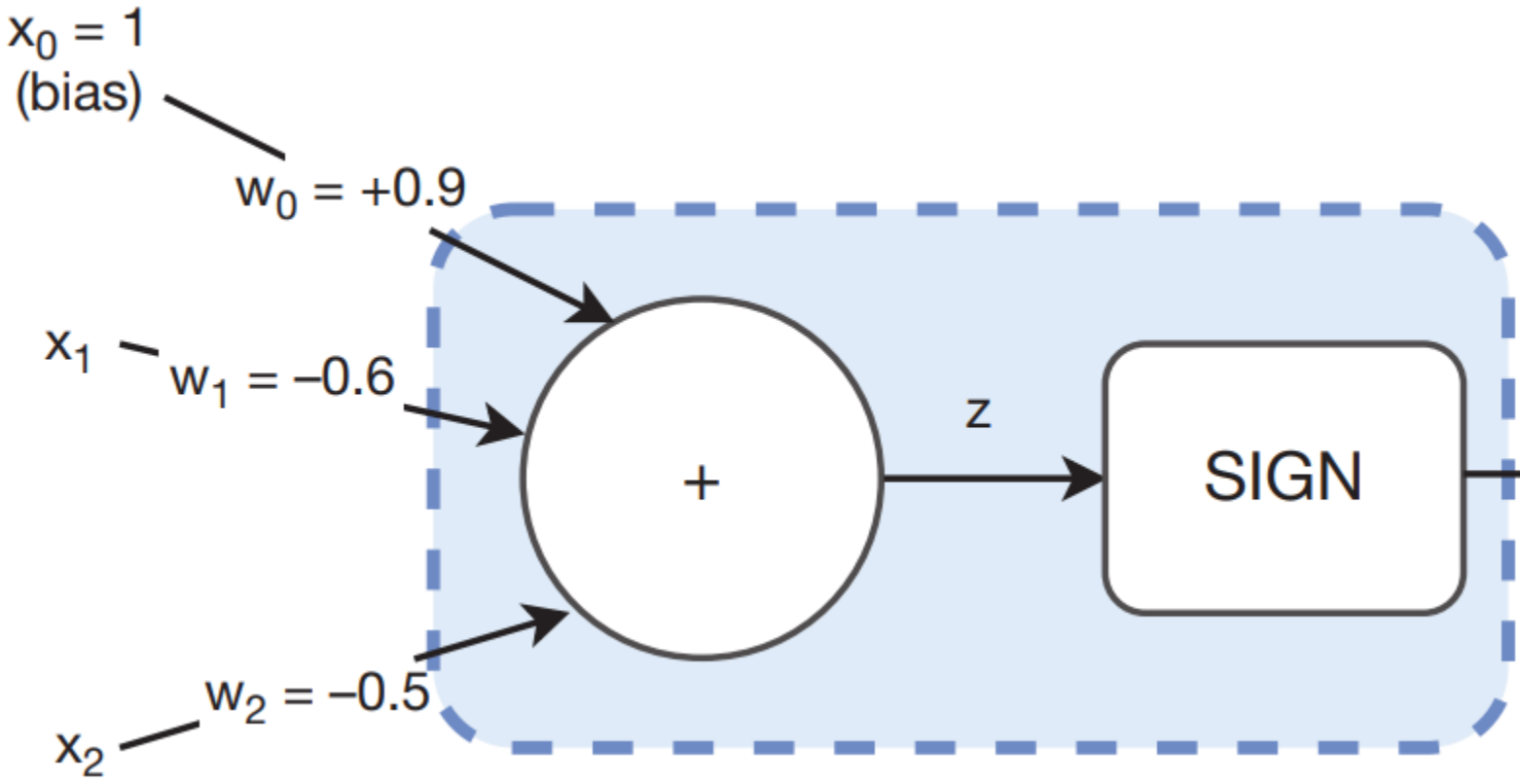
$$z = \sum_{i=0}^{n} w_i x_i$$

Key Components Cont'd

- **Activation Function (Sign Function or Threshold Function):** The weighted sum is then passed through an activation function, traditionally a sign function or threshold function. The output is binary, usually -1 or 1, depending on whether the weighted sum is above or below a certain threshold. The sign function can be defined as:

$$f(z) = \begin{cases} -1, & z < 0 \\ 1, & z \geq 0 \end{cases}$$

Two Input Perceptron Example



$$y = f(z), \text{ where}$$

$$z = \sum_{i=0}^{n} w_i x_i$$

$$f(z) = \begin{cases} -1, & z < 0 \\ 1, & z \geq 0 \end{cases}$$

$$x_0 = 1 \text{ (bias term)}$$

Two Input Perceptron Example Cont'd

X_0	X_1	X_2	W_0*X_0	W_1*X_1	W_2*X_2	Z	Y
1	–1 (False)	–1 (False)	0.9	0.6	0.5	2.0	1 (True)
1	1 (True)	–1 (False)	0.9	–0.6	0.5	0.8	1 (True)
1	–1 (False)	1 (True)	0.9	0.6	–0.5	1.0	1 (True)
1	1 (True)	1 (True)	0.9	–0.6	–0.5	–0.2	–1 (False)

- We know this set of weights W=[0.9,-0.6,-0.5] implement NAND gate. What if we are given random weights, is there a way to find optimal weights that predicts the output similar to the actual output?
- Multiple sets of weights will help the perceptron in separating the two classes, but will they be optimal?
- How to find optimal weights that classify all the examples correctly?

Perceptron Learning Algorithm

- The perceptron learning algorithm is a simple supervised learning algorithm designed for binary classification tasks.
- During the training process it updates the weights based on the difference between the predicted output and the true output.

Perceptron Learning Algorithm Cont'd

1. Randomly initialize the weights.
2. Select one input/output pair at random.
3. Present the values $x_1, \ldots, x_n$ to the perceptron to compute the output y.
4. If the output y is different from the ground truth for this input/output pair, adjust the weights in the following way:
   a. If $y < 0$, add ηx_i to each w_i.
   b. If $y > 0$, subtract ηx_i from each w_i.
5. Repeat steps 2, 3, and 4 until the perceptron predicts all examples correctly.

Updating Weights using Perceptron Learning Algorithm

Given the inputs $\mathbf{X_i}$, random weights $\mathbf{W_i}$, labels $\mathbf{Y_{Actual}}$, and learning rate η, compute optimal set of weights using perceptron learning algorithm (PLA) such that the f(z) which is $\mathbf{Y_{predicted}} = \mathbf{Y_{actual}}$

```
w = [0.2, -0.6, 0.25]      LEARNING_RATE = 0.1
```

$$W_i = W_i + (\eta . Y_{Actual} . X_i)$$

X_0	X_1	X_2	Y_{Actual}
1	-1	-1	1
1	-1	1	1
1	1	-1	1
1	1	1	-1

$$y = f(z), \text{ where}$$

$$z = \sum_{i=0}^{n} w_i x_i$$

$$f(z) = \begin{cases} -1, & z < 0 \\ 1, & z \geq 0 \end{cases}$$

$$x_0 = 1 \text{ (bias term)}$$

NAND gate implementation

X_0	X_1	X_2	Y_{actual}
1	-1	-1	1
1	-1	1	1
1	1	-1	1
1	1	1	-1

$W = [\overset{w_0}{0.2}, \overset{w_1}{-0.6}, \overset{w_2}{0.25}]$, $\eta = 0.1$

$$Z = \sum_{i=0}^{n} w_i x_i$$

$Z = w_0 x_0 + w_1 x_1 + w_2 x_2$

$$f(z) = \begin{cases} 1 \text{ if } z \geq 0 \\ -1 \text{ otherwise} \end{cases}$$

$Y_{Predicted}$

$Z_1 = (\overset{w_0}{0.2})(\overset{x_0}{1}) + (\overset{w_1}{-0.6})(\overset{x_1}{-1}) + (\overset{w_2}{0.25})(\overset{x_2}{-1}) = 0.55 > 0 \Rightarrow 1$

$Z_2 = (0.2)(1) + (-0.6)(-1) + (0.25)(1) = 1.05 > 0 \Rightarrow 1$

$Z_3 = (0.2)(1) + (-0.6)(1) + (0.25)(-1) = -0.65 < 0 \Rightarrow -1$ (Misclassification)

$Z_4 = (0.2)(1) + (-0.6)(1) + (0.25)(1) = -0.15 < 0 \Rightarrow -1$

Using weight update rule of PLA on Z_3 $\Rightarrow$ (Iteration 1 of weight update)

$w_i = w_i + \eta\, Y_{actual}\, X_i$

$w_0 = w_0 + \eta\, Y_{Actual}\, X_0$: $w_0 = 0.2 + (0.1)(1)(1)$; $w_0 = 0.3$

$w_1 = w_1 + \eta\, Y_{Actual}\, X_1$: $w_1 = -0.6 + (0.1)(1)(1)$; $w_1 = -0.5$

$w_2 = w_2 + \eta\, Y_{Actual}\, X_2$: $w_2 = 0.25 + (0.1)(1)(-1)$; $w_2 = 0.15$

Now, the updated weights are $W = [\overset{w_0}{0.3}, \overset{w_1}{-0.5}, \overset{w_2}{0.15}]$

Use these updated weights to implement NAND gate

$Z = w_0 x_0 + w_1 x_1 + w_2 x_2$ $\quad Y_{Predicted}$

$Z_1 = (0.3)(1) + (-0.5)(-1) + (0.15)(-1) = 0.65 > 0 \Rightarrow 1$

$Z_2 = (0.3)(1) + (-0.5)(-1) + (0.15)(1) = 0.95 > 0 \Rightarrow 1$

$Z_3 = (0.3)(1) + (-0.5)(1) + (0.15)(-1) = -0.35 < 0 \Rightarrow -1$ (Again misclassification)

$Z_4 = (0.3)(1) + (-0.5)(1) + (0.15)(1) = -0.05 < 0 \Rightarrow -1$

$\Rightarrow$ (Iteration 2 of weight update)

$w_0 = w_0 + \eta\, Y_{actual}\, X_0$: $w_0 = 0.3 + (0.1)(1)(1)$; $w_0 = 0.4$

$w_1 = w_1 + \eta\, Y_{actual}\, X_1$: $w_1 = (-0.5) + (0.1)(1)(1)$; $w_1 = -0.4$

$w_2 = w_2 + \eta\, Y_{actual}\, X_2$: $w_2 = (0.15) + (0.1)(1)(-1)$; $w_2 = 0.05$

Now the updated weights from iteration 2 are :-

$W = [\overset{w_0}{0.4}, \overset{w_1}{-0.4}, \overset{w_2}{0.05}]$

Now again calculating the $Y_{Predicted}$ with updated weights

$Z = w_0 x_0 + w_1 x_1 + w_2 x_2$ $\quad Y_{Predicted}$

$Z_1 = (0.4)(1) + (-0.4)(-1) + (0.05)(-1) = 0.75 > 0 \Rightarrow 1$

$Z_2 = (0.4)(1) + (-0.4)(-1) + (0.05)(1) = 0.85 > 0 \Rightarrow 1$

$Z_3 = (0.4)(1) + (-0.4)(1) + (0.05)(-1) = -0.05 < 0 \Rightarrow -1$ (Misclassification)

$Z_4 = (0.4)(1) + (-0.4)(1) + (0.05)(1) = 0.05 > 0 \Rightarrow 1$ (Misclassification)

(considering this example of misclassification for weight update) $\Rightarrow$ (Iteration 3 of weight update)

* Again applying weight update rule :

$w_0 = w_0 + \eta\, Y_{actual}\, X_0$: $w_0 = (0.4) + (0.1)(-1)(1)$; $w_0 = 0.3$

$w_1 = w_1 + \eta\, Y_{actual}\, X_1$: $w_1 = -0.4 + (0.1)(-1)(1)$; $w_1 = -0.5$

$w_2 = w_2 + \eta\, Y_{actual}\, X_2$: $w_2 = 0.05 + (0.1)(-1)(1)$; $w_2 = -0.05$

Now, the updated weights are : $W = [\overset{w_0}{0.3}, \overset{w_1}{-0.5}, \overset{w_2}{-0.05}]$

Check if these weights correctly implement the NAND gate

$Z = w_0 x_0 + w_1 x_1 + w_2 x_2$ $\quad Y_{Predicted}$

$Z_1 = (0.3)(1) + (-0.5)(-1) + (-0.05)(-1) = 0.85 > 0 \Rightarrow 1$

$Z_2 = (0.3)(1) + (-0.5)(-1) + (-0.05)(1) = 0.75 > 0 \Rightarrow 1$

$Z_3 = (0.3)(1) + (-0.5)(1) + (-0.05)(-1) = -0.15 < 0 \Rightarrow -1$ (Misclassification)

$Z_4 = (0.3)(1) + (-0.5)(1) + (-0.05)(1) = -0.25 < 0 \Rightarrow -1$

* Again update weight to remove misclassification $\Rightarrow$ (Iteration 4 of weight update)

$w_0 = w_0 + \eta\, Y_{actual}\, X_0$: $w_0 = 0.3 + (0.1)(1)(1)$; $w_0 = 0.4$

$w_1 = w_1 + \eta\, Y_{actual}\, X_1$: $w_1 = -0.5 + (0.1)(1)(1)$; $w_1 = -0.4$

$w_2 = w_2 + \eta\, Y_{actual}\, X_2$: $w_2 = -0.05 + (0.1)(1)(-1)$; $w_2 = -0.15$

M.M.

Now, the updated weights from iteration 4 are:

$w_0 \quad w_1 \quad w_2$

$w = [0.4, -0.4, -0.15]$

Check if these weights are optimal and result in no misclassification:

$z = w_0 x_0 + w_1 x_1 + w_2 x_2$

		$Y_{predicted}$	Y_{Actual}
z_1	$(0.4)(1) + (-0.4)(-1) + (-0.15)(-1) = 0.95 > 0 \Rightarrow$	1	1
z_2	$(0.4)(1) + (-0.4)(-1) + (-0.15)(1) = 0.65 > 0 \Rightarrow$	1	1
z_3	$(0.4)(1) + (-0.4)(1) + (-0.15)(-1) = 0.15 > 0 \Rightarrow$	1	1
z_4	$(0.4)(1) + (-0.4)(1) + (-0.15)(1) = -0.15 < 0 \Rightarrow$	-1	-1

All the misclassifications are removed, hence applying PLA for weight update in four iterations resulted in optimal set of weights i.e., $w = [0.4, -0.4, -0.15]$.

We started with randomly selected weights as follows:

$w = [0.2, -0.6, 0.25]$

Then iteratively updated weights until $Y_{predicted} = Y_{Actual}$.

$w_0 \quad w_1 \quad w_2$

Iteration 1 updated weights: $w = [0.3, -0.5, 0.15]$

Iteration 2 updated weights: $w = [0.4, -0.4, 0.05]$

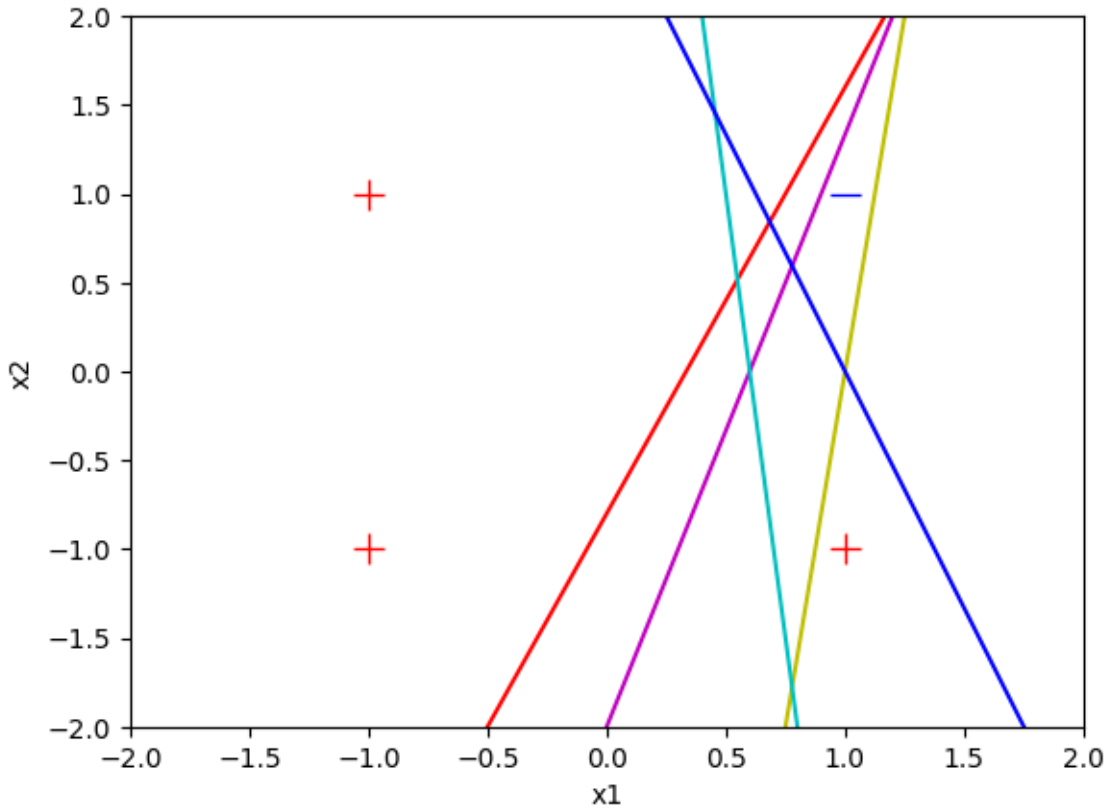
Iteration 3 updated weights: $w = [0.3, -0.5, -0.05]$

Iteration 4 updated weights: $w = [0.4, -0.4, -0.15]$

Decision boundary with various set of weights

Learning process progressing in the following order:

1. Initially set random weights: Red
2. After first weight update: Magenta (iteration 1)
3. After second weight update: Green (iteration 2)
4. After third weight update: Cyan (iteration 3)
5. After fourth weight update: blue (iteration 4)



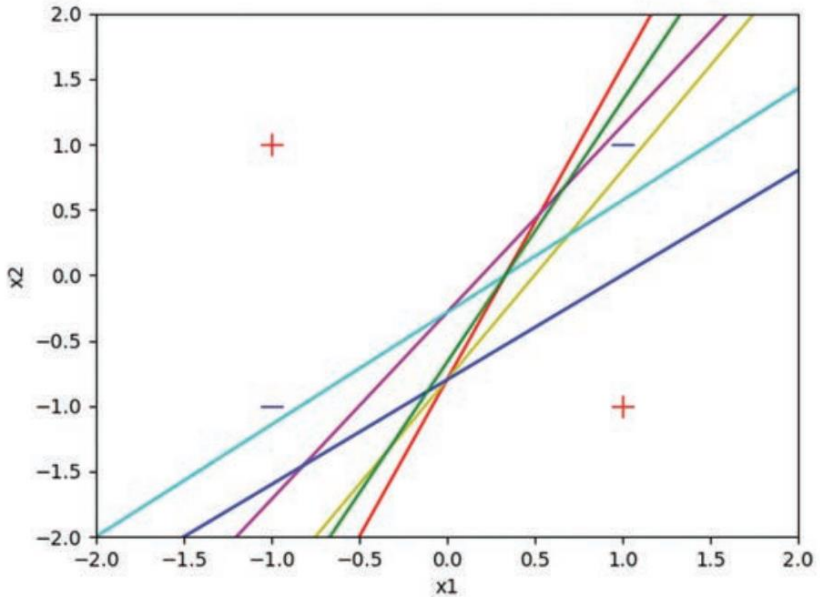
Limitations of the Perceptron

1. **Linear separability requirement:**
   Can only learn and represent linearly separable functions.
2. **Binary output:**
   Produces binary outputs (0 or 1), limiting its ability to model complex relationships.
3. **Inability to learn non-linear functions:**
   Incapable of learning non-linear functions, making it unsuitable for tasks like XOR.
4. **Sensitivity to input noise:**
   Prone to sensitivity to small variations or noise in input data.
5. **Lack of hidden layers:**
   Single-layer structure limits its capacity to capture complex features and hierarchies.
6. **No mechanism for weight update adaptation:**
   Original learning rule lacks adaptive mechanisms for learning rates or efficient weight updates.
7. **Limited representational power:**
   Struggles with representing functions requiring many weights or complex transformations.

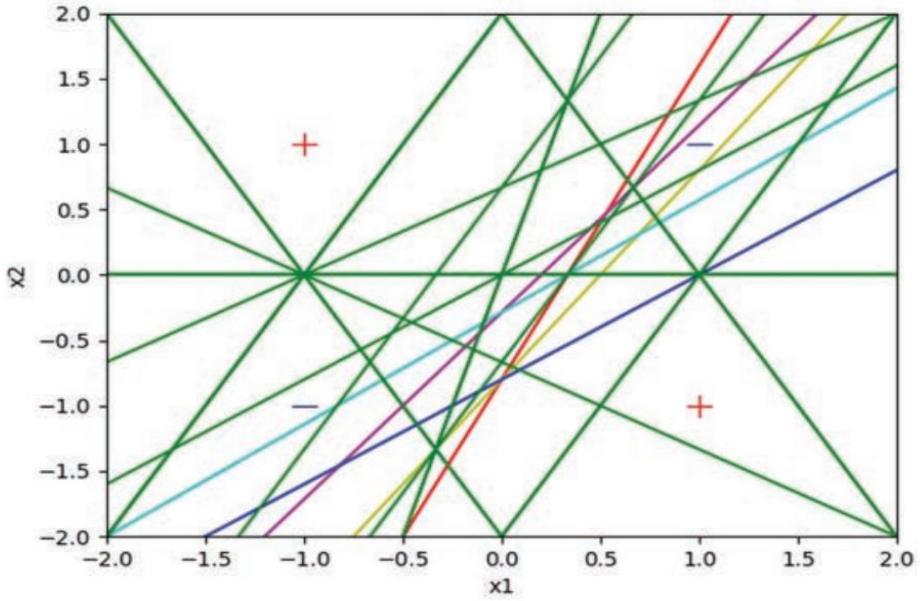
Limitations of the Perceptron: XOR

X_0	X_1	Y
False	False	False
True	False	True
False	True	True
True	True	False

After 6 weight adjustments

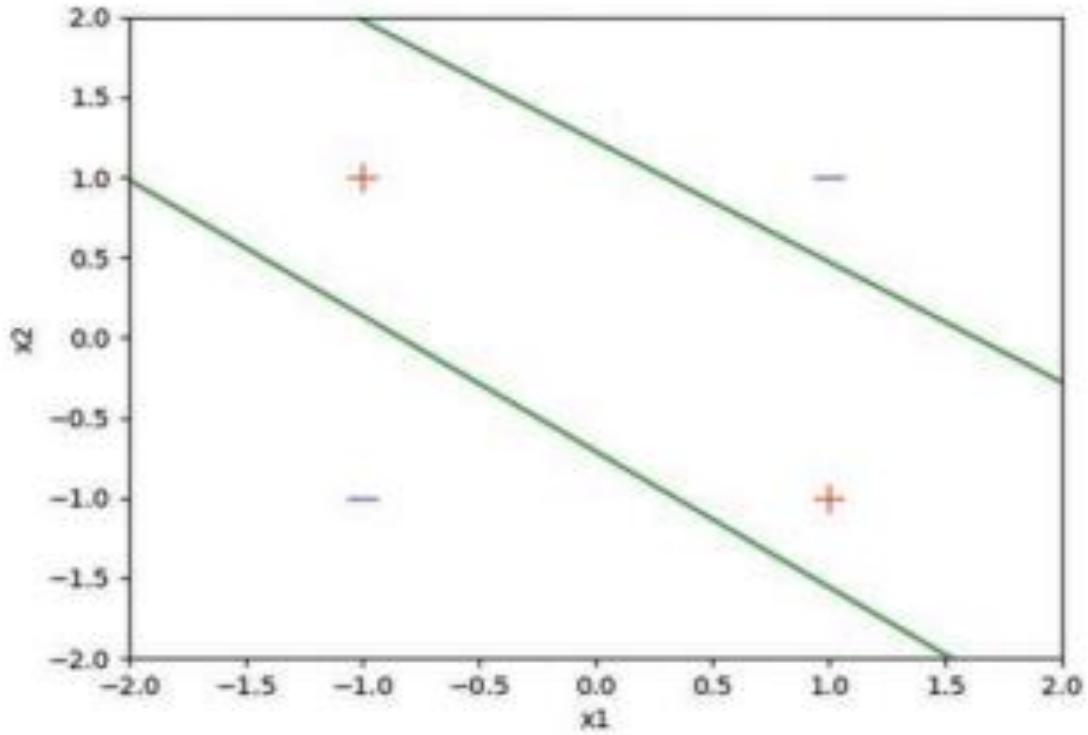


After 30 weight adjustments



Solution?

Combining Multiple Perceptrons



Combining Multiple Perceptrons Cont'd

$$\left(\overline{X_1 \,.\, X_2}\right).(X_1 + X_2)$$

$\left(\overline{X_1.X_2}\right).(X_1 + X_2)$

X_0	X_1	X_2	$Y_0\ (\overline{X_1.X_2})$	$Y_1\ (X_1 + X_2)$	Y_2
1	–1 (False)	–1 (False)	1.0	–1.0	–1.0 (False)
1	1 (True)	–1 (False)	1.0	1.0	1.0 (True)
1	–1 (False)	1 (True)	1.0	1.0	1.0 (True)
1	1 (True)	1 (True)	–1.0	1.0	–1.0 (False)

Combining Multiple Perceptrons Cont'd